

CATT: Character-based Arabic Tashkeel Transformer

Faris Alasmary
Abjad Ltd.
falasmary@abjad.com.sa

Orjuwan Zaafarani
Abjad Ltd.
oturki@abjad.com.sa

Ahmad Ghannam
Abjad Ltd.
aghannam@abjad.com.sa

Abstract

Tashkeel, or Arabic Text Diacritization (ATD), greatly enhances the comprehension of Arabic text by removing ambiguity and minimizing the risk of misinterpretations caused by its absence. It plays a crucial role in improving Arabic text processing, particularly in applications such as text-to-speech and machine translation. This paper introduces a new approach to training ATD models. First, we finetuned two transformers, encoder-only and encoder-decoder, that were initialized from a pretrained character-based BERT. Then, we applied the Noisy-Student approach to boost the performance of the best model. We evaluated our models alongside 11 commercial and open-source models using two manually labeled benchmark datasets: WikiNews and our CATT dataset. Our findings show that our top model surpasses all evaluated models by relative Diacritic Error Rates (DERs) of 30.83% and 35.21% on WikiNews and CATT, respectively, achieving state-of-the-art in ATD. In addition, we show that our model outperforms GPT-4-turbo on CATT dataset by a relative DER of 9.36%. We open-source our CATT models and benchmark dataset for the research community¹.

1 Introduction

The Arabic language is characterized by its rich morphology and complex syntactic structure. One of the unique features of Arabic is the use of diacritics or Tashkeel, which are small marks above or below the letters that indicate vowels or other phonetic aspects of pronunciation. These diacritics are favorable for understanding the meaning of words, as their absence can lead to ambiguities and misinterpretations. They are also crucial in improving performance of applications such as text-to-speech and machine translation (Fadel et al., 2019b). The diacritics can be affected by the context of the sentence as shown in the following example:

¹<https://github.com/abjadai/catt>

1. سَاقُ الرَّجُلِ السَّيَّارَةِ

Translation: The man drove the car.

2. سَاقُ الرَّجُلِ بِهَا جُرُوحٌ

Translation: The man's leg has wounds.

In the first sentence, "سَاقُ" or "Saqa" is interpreted as "drove", denoting an action. However, when the context changes, the same word, now pronounced as "Sagu" takes on a completely different meaning, becoming a noun that translates to "leg". This change highlights the crucial role of diacritics in quickly clarifying the meanings of sentences, as the characters remain the same. At the same time, the pronunciation varies depending on the context. In the previous example, the meaning of the word "سَاقُ" can be comprehended even without diacritics, as long as the reader considers the complete sentence and understands the context provided by the surrounding words. This fact raises the following question: "Will a pretrained BERT model help in improving the ATD models?"

In this paper, we propose a training strategy based on a pretrained character-based BERT (Kenton and Toutanova, 2019; Liu et al., 2019), and a self-training approach called Noisy-Student (NS) (Xie et al., 2020). Throughout the paper, we will answer the following research questions:

- **RQ1:** Does the ATD model benefit from Masked Language Model (MLM) pretraining?
- **RQ2:** Does training ATD model for more iterations help?
- **RQ3:** Is the NS approach effective in ATD models?

2 Related Work

Previous research has investigated a broad spectrum of approaches to address the diacritization task, beginning with rule-based methods, moving to classical machine learning models, and reaching

sophisticated deep learning architectures (Almanea, 2021). In addition, comprehensive experiments show that deep learning methods outperform non-neural techniques, particularly when substantial training data is available (Fadel et al., 2019a).

Fadel et al. (2019b) tested a refined version of the Tashkeela dataset (Zerrouki and Balla, 2017; Fadel et al., 2019a) using the Shakkala² model. They also trained a character-level RNN with a Block-Normalized Gradient (BNG) module. The BNG technique normalizes gradients within each batch, potentially speeding up training and improving generalization (Yu et al., 2017).

Abbad and Xiong’s (2020) ATD approach consisted of a three-part pipeline: a multi-layer LSTM and dense layers, a character-level rule-based corrector for specific error correction, and a word-level statistical corrector that leveraged context and distance information to resolve diacritization issues. Furthermore, they developed an enhanced version of the system and named it Multilevel Diacritizer (Abbad and Xiong, 2021).

Madhfar and Qamar (2020) implemented 3 different ATD models. The first one was a baseline model consisting of 3 deep Bidirectional Long Short-Term Memory (BiLSTM) layers. The second model was an encoder-decoder with 3 LSTM layers for the encoder and 2 LSTM layers for the decoder. The last model was based on Tacotron encoder (Wang et al., 2017) that uses CBHG module (Lee et al., 2017).

AlKhamissi et al. (2020) proposed two architectures: the Two-Level Diacritizer (D2) and the Two-Level Diacritizer with Decoder (D3). D3 builds upon the capabilities of D2 by accepting partially diacritized text as input. These models have a word-level encoder as well as a character-level encoder. The results of both encoders are combined by an attention mechanism and fed to a unidirectional LSTM layer to predict diacritics.

Darwish et al. (2021) created two Deep Neural Networks (DNNs); the first utilizes a character-based BiLSTM model with unique features for each character, while the second uses a word-level BiLSTM layer and a subsequent dense layer with Softmax activation.

Karim and Abandah (2021) studied the effect of varying the training dataset size. Each time, they trained a BiLSTM model and evaluated its performance. The results demonstrated that error

rates improve as the size of the training corpus increases.

Al-Rfooh et al. (2023) finetuned a token-free multilingual model called ByT5 (Xue et al., 2022) to perform Arabic text diacritization as a sequence-to-sequence task, similar to the translation task.

Recently, Skiredj and Berrada (2024) introduced the Pre-FineTuned Token Classification for Arabic Diacritization (PTCAD) model. This approach treats Arabic text diacritization as a downstream task for a pretrained BERT-like model. The approach starts with a pretraining phase on linguistically relevant tasks, such as Part-of-Speech (POS) tagging and Segmentation, which are framed as Masked Language Modelling (MLM) tasks. This pretraining helps enhance the model’s contextual understanding. Then, it moves into a finetuning phase where diacritization is handled as a token classification task. This phase leverages the contextual insights gained earlier to enhance diacritization accuracy.

Unlike Skiredj and Berrada’s (2024) method, we consider a simpler approach where we directly pre-train a character-level BERT model with no further modifications or extra labeling.

3 Dataset Preparation

3.1 Training Data

As shown by Karim and Abandah (2021), training on larger dataset improves the performance of the ATD model. We used the whole Tashkeela dataset (Zerrouki and Balla, 2017) for training which consists of 1,658,325 samples. Initially, we filtered out samples that had fewer than 6 characters or more than 1024 characters, considering both letters and diacritics as characters. Next, we removed samples with a Diacritics-to-Letters (DTL) ratio of less than 60%. We defined this ratio as follows:

$$DTL\ ratio = \frac{\# \text{ of diacritics}}{\# \text{ of letters}}$$

In addition, we performed a cleaning process on each sentence in the filtered list, removing non-Arabic characters. This includes special characters, English letters, Arabic and Indian numerals, as well as punctuation marks in both English and Arabic. After this cleaning process, the total number of remaining samples was 1,330,539.

To pretrain the character-based BERT, we scraped 18,543,025 data samples from various sources, including X and online news websites.

²<https://github.com/Barqawiz/Shakkala>

Training on this data will help the model to understand the Modern Standard Arabic (MSA) as well as the colloquial dialects. To align with the architectural requirements of the ATD models for subsequent finetuning, we capped the maximum sequence length of the model at 1024 characters. However, we set the maximum length of the training sentences during MLM pretraining to 512. Consequently, all samples in the pretraining data were truncated at the last space character when the length of the sample exceeds 512 to preserve the context of the last word in the sample.

3.2 Benchmark Data

The Tashkeela (Zerrouki and Balla, 2017) dataset contains data from different sources, including both MSA and classical Arabic. Around 98.85% of the Tashkeela dataset consists of content obtained from 97 books found in the Shamila³ library. The Shamila library is an Islamic electronic library with hundreds of works covering Hadith, Fiqh, history, preaching, Islamic rules, and Arabic language (Zerrouki and Balla, 2017). It can be misleading to assess ATD models using a portion of this dataset for the following reasons:

1. Most of the dataset’s books contain partial or complete citations from the Holy Quran and Hadith as well as from each other, which might lead to data contamination eventually impacting the evaluation results.
2. Most resources in the dataset are written in classical Arabic. However, when evaluating ATD models for today’s applications such as text-to-speech or machine translation, relying only on this dataset may lead to unreliable results. This is because the target users of these applications typically use MSA or colloquial dialects, which differ from classical Arabic.

As a result, we created the CATT benchmark dataset. This dataset comprises 742 sentences, which we scraped from an internet news source in 2023. It covers multiple topics including science and technology, economics, politics, sports, arts, and culture. The CATT dataset was manually diacritized by two expert native Arabic speakers and then validated by a third expert. This dataset contains names of people and places in both Arabic and English. As for the English names, they

³<https://shamela.ws>

Class Name	Diacritic
Fatha	َ
Kasra	ِ
Dhamma	ُ
Tanween Fath	َْ
Tanween Kasr	ِْ
Tanween Dhamm	ُْ
Shadda	ّ
Shadda + Fatha	َّ
Shadda + Kasra	ِّ
Shadda + Dhamma	ُّ
Shadda + Tanween Fath	َّْ
Shadda + Tanween Kasr	ِّْ
Shadda + Tanween Dhamm	ُّْ
Sukoon	◌ْ◌ْ◌ْ
No Tashkeel	<NT>

Table 1: Arabic Diacritics

are written in Arabic letters and diacritized based on their pronunciation. Also, the numbers in the sentences are written in textual form rather than the numeric form. This helps in evaluating the models without the need for a text normalizer (TN).

Moreover, we used WikiNews (Darwish et al., 2017) benchmark dataset to evaluate all models. This dataset comprises 400 manually diacritized MSA sentences. It covers multiple topics, most of which are similar to CATT’s topics, from the years 2013 and 2014.

Data	Chars	Words	Lines
BERT Pretraining	2.06B	359.96M	18.54M
Tashkeela	213.86M	42.43M	1.33M

Table 2: Data Summary (After Preparation)

4 Experiments

We pretrained a character-based BERT model to find the effect of MLM pretraining on the diacritization performance. The model has 6 layers with $d_{model} = 512$ and number of heads = 16. The model was trained using MLM loss for 6 epochs with a batch size of 512, using the data shown in Table 2.

For ATD models, we selected two transformer architectures: Encoder-Decoder (ED) with 3 layers and Encoder-Only (EO) with 6 layers. We set the number of layers to 3 in the ED model to ensure comparability with the EO model in terms of the total number of layers. All our ATD models were trained with the following configurations:

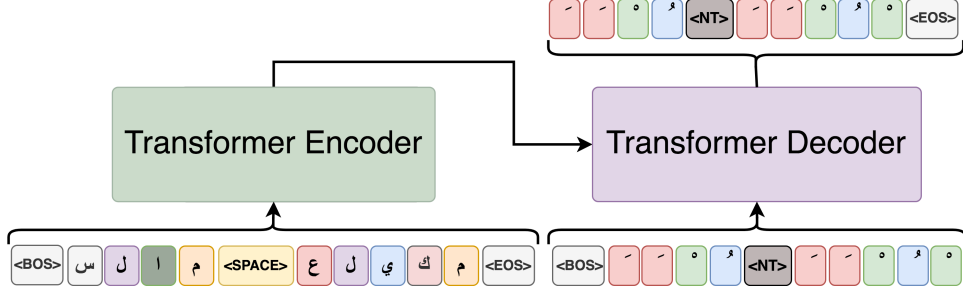


Figure 1: Encoder-Decoder (ED) Model

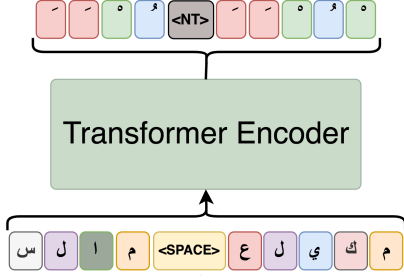


Figure 2: Encoder-Only (EO) Model

$d_{model} = 512$, number of heads = 16, batch size = 32, and dropout = 10%. We used the AdamW optimizer (Loshchilov and Hutter, 2018) with a learning rate of 3×10^{-5} and a weight decay of 1×10^{-2} . Each model was trained on a single dedicated A100 GPU. All ATD models in our experiments were trained for a maximum of 200 epochs with an early stopping criteria.

Generally, we define the input text X with a total number of characters T as a series of characters $x_1, x_2, x_3, \dots, x_T$ where each character represents an undiacritized Arabic letter. Correspondingly, the output sequence Y consists of diacritics $y_1, y_2, y_3, \dots, y_T$ with each diacritic y_i associated with the respective letter x_i . In EO models, we express the relationship between letters and diacritics as follows:

$$P(y_i | x_1, \dots, x_T)$$

In other words, we predict the diacritic y_i conditioned only on the input text $x_1, \dots, x_T$. In ED models, on the other hand, we consider the ATD task as a translation task, where the input text represents the source language and the output diacritics sequence represents the target language. We express the relationship between letters and diacritics in ED models as follows:

$$P(y_i | x_1, \dots, x_T, y_1, \dots, y_{i-1})$$

where the diacritic y_i is conditioned on both the input text $x_1, \dots, x_T$ and the previous diacritics $y_1, \dots, y_{i-1}$. In fact, native Arabic speakers rely on both the textual content and diacritics to better disambiguate the intended meaning of the sentence. For example, the sentence *اشترت لعبة* is a complete and valid Arabic sentence that can be interpreted as "I bought a toy" or "A toy was bought". The only way to differentiate between them in the textual form is by adding diacritics as follows: *اَشْتَرَيْتُ لُعْبَةً* which means "I bought a toy" or *اُشْتَرِيَتْ لُعْبَةٌ* which means "A toy was bought". In both cases, the diacritics of the second word heavily depend on the diacritics of the first word. Therefore, by conditioning on both the input text and the previous diacritics, the model can achieve better performance. Figure 1 and Figure 2 show both transformer architectures.

Our experiments involved training a total of four models using both the ED and EO architectures. For both architectures, we used the pretrained character-based BERT as the basis for initializing the weights. Consequently, we obtained one ED model and one EO model with weights that reflect the knowledge encoded in the pretrained BERT. Additionally, we trained the other two models, consisting of one ED model and one EO model, where the weights were randomly initialized. These models allowed us to explore the impact of different weight initialization strategies on the performance and behavior of the models.

For each model in our experiments, we selected two checkpoints. The first checkpoint was chosen after training the model for 5 epochs, while the second checkpoint was chosen as the best checkpoint achieved after training for a longer period. The best checkpoint of ED model was at epoch 175 while the best checkpoint of EO was at epoch 192. The purpose of this selection process was to study the

impact of extended training duration on the models, even when they were exposed to the same amount of data.

Moreover, we randomly sampled 1M sentences from the pretraining dataset to be pseudo-labeled using the NS (Xie et al., 2020) technique. We used the best checkpoints of both ED and EO models to pseudo-label two copies of the sampled data. Finally, a new ED model as well as a new EO model were trained on Tashkeela data combined with the pseudo-labeled 1M sentences. Both models’ parameters were initialized from the best checkpoints. Table 3 shows the details of the combined dataset after the filtration process described in section 3.1.

Data	Chars	Words	Lines
Tashkeela	213.86M	42.43M	1.33M
Tashkeela + NS (ED)	292.01M	56.21M	2.22M
Tashkeela + NS (EO)	290.59M	55.95M	2.20M

Table 3: The combined datasets using NS pseudo-labeling by both ED and EO models (After Preparation).

5 Results

There are two methods to evaluate ATD models: one with Case Ending (CE) and one without Case Ending (No CE). In the No CE approach, the diacritic on the last letter is excluded during performance evaluation, while the CE approach includes the diacritic in the evaluation. The presence or absence of the diacritic on the last letter mostly depends on grammatical rules. Error rates without CE reflect the performance specifically on the core word, while error rates with CE represent the overall performance of the model (Madhfar and Qamar, 2020).

We compared our models with 9 models, namely, CBHG (Madhfar and Qamar, 2020), Sakhr⁴, Farasa (Darwish and Mubarak, 2016), D2 and D3 models (AlKhamissi et al., 2020), Alkhalil Tashkeel⁵, Mishkal⁶, Multilevel Diacritizer (Abbad and Xiong, 2021), and Shakkala. Moreover, we compared the performance of our models with the performance of two Large Language Models (LLMs) on the ATD task: GPT-4-turbo⁷, and Command R+⁸. During evaluation, all preprocessing steps are applied to both the reference text and the output of all models

to ensure fair comparisons. In the case of long sentence diacritization, we follow a process of splitting the sentence into smaller segments based on punctuation. Each small sentence is then diacritized individually, and finally, the segments are combined to reconstruct the original text with diacritics.

The results in Tables 4 and 5 show that our models achieved state-of-the-art performance in the ATD task, outperforming all 11 models. Our ED model achieved the lowest DER and Word Error Rate (WER) on the CATT benchmark dataset with and without CE. Moreover, both ED and EO models achieved low DER and WER scores on the WikiNews benchmark dataset compared to other ATD models.

The evaluation of GPT-4 on the WikiNews dataset shows a high level of performance. However, when GPT-4 is evaluated on the CATT dataset, its performance appears to be comparatively normal. This difference in performance can likely be attributed to the fact that GPT-4 was trained on web data predating December 2023. Based on the observed significant performance gap between testing GPT-4 on CATT and WikiNews, we believe that GPT-4 was most likely trained on the WikiNews dataset. It is worth noting that the WikiNews dataset was published in 2017 (Darwish et al., 2017), while the CATT dataset was created in March 2024.

CATT Benchmark Dataset				
Model	CE (%)		No CE (%)	
	DER	WER	DER	WER
CBHG	10.808	42.680	8.313	34.386
Command R+	13.169	48.518	11.329	44.158
GPT-4	9.515	38.311	8.113	33.505
Sakhr	13.841	56.661	11.125	47.993
Farasa	17.825	65.783	15.414	60.114
D2	13.310	49.417	10.036	38.391
D3	58.313	98.710	48.018	95.186
Alkhalil	14.232	53.413	11.568	45.777
Mishkal	16.482	60.844	10.796	40.215
Multilevel	16.503	58.076	13.434	50.147
Shakkala	13.494	50.387	10.386	40.643
EO (Ours)	8.805	35.597	7.117	29.759
ED (Ours)	8.624	34.191	6.989	28.477

Table 4: Benchmark results on CATT dataset.

5.1 RQ1: Does the ATD model benefit from MLM pretraining?

Tables 6 through 11 detail our experiments’ results on the CATT and WikiNews datasets. The experiments clearly show the advantages of MLM pre-training since it consistently boosted performance

⁴<https://tashkeel.alsharekh.org>

⁵<https://tashkeel.alkhalilarabic.com>

⁶<https://tahadz.com/mishkal>

⁷<https://chatgpt.com>

⁸<https://huggingface.co/CohereForAI/c4ai-command-r-plus>

WikiNews Benchmark Dataset				
Model	CE (%)		No CE (%)	
	DER	WER	DER	WER
CBHG	8.276	36.032	5.448	21.528
Command R+	21.470	54.335	17.755	49.611
GPT-4	0.551*	2.276*	0.326*	1.024*
Sahkr	7.843	38.413	5.628	29.921
Farasa	19.584	69.536	17.752	66.601
D2	9.231	32.622	6.164	21.744
D3	58.558	109.028	48.234	99.408
Alkhalil	14.912	46.793	12.145	35.699
Mishkal	15.246	54.187	8.558	27.337
Multilevel	12.431	45.054	9.318	36.217
Shakkala	9.978	37.241	6.593	24.988
EO (Ours)	5.425	22.132	3.105	12.679
ED (Ours)	6.070	20.745	3.744	11.982

Table 5: Benchmark results on WikiNews dataset.

across all tested models, regardless of the number of training steps, the CE conditions, or the used benchmark dataset. Table 6 indicates that initializing the encoder part of the ED model with pretrained MLM weights boosted the performance by a relative ratio of 12.28% when evaluated on the CATT dataset. Similarly, Table 7 shows that initializing the EO model with pretrained MLM weights improved the performance by a relative ratio of 15.94% when evaluated on the WikiNews dataset. Our results indicate that weight initialization from pretrained MLM weights can boost the performance of both ED and EO models compared to random initialization.

5.2 RQ2: Does training ATD model for more iterations help?

By analyzing the results presented in Tables 6 and 7, and Tables 8 and 9, we can see that the training for more iterations consistently enhances model performance across all metrics, CE conditions, and datasets. It is notable that for few training iterations, the EO model outperforms the ED model on the CATT benchmark dataset as shown in Tables 6 and 8. In the other hand, when we trained both models for more iterations, the ED model surpasses the EO model in all performance metrics on the same dataset. Moreover, Tables 7 and 9 show the performance of both models on the WikiNews dataset. They demonstrate that with fewer training steps, the EO model performs better in terms of DER and WER under both CE and No CE conditions. However, only the DER for the EO model was better than the DER of the ED model as we trained them for more iterations. The performance difference between EO and ED could be attributed to the vari-

ation in model architecture. Specifically, the EO model had all 6 layers initialized using pretrained MLM weights, while in ED, only the 3 layers of the encoder part were initialized with pretrained MLM weights. Generally, our experiments show that training for more iterations can improve the ATD model performance.

CATT Benchmark Dataset				
Model	CE (%)		No CE (%)	
	DER	WER	DER	WER
EO – From Scratch	9.613	38.685	7.631	31.610
EO – MLM	9.260	37.492	7.411	30.969
ED – From Scratch	10.359	39.753	8.003	31.654
ED – MLM	9.087	35.757	7.272	29.483

Table 6: The impact of MLM pretraining vs. training from scratch, after training for *more* iterations when evaluated on CATT benchmark dataset.

WikiNews Benchmark Dataset				
Model	CE (%)		No CE (%)	
	DER	WER	DER	WER
EO – From Scratch	7.009	25.857	4.527	16.761
EO – MLM	5.892	23.785	3.469	14.116
ED – From Scratch	7.271	24.870	4.464	14.202
ED – MLM	6.376	21.954	4.001	12.926

Table 7: The impact of MLM pretraining vs. training from scratch, after training for *more* iterations when evaluated on WikiNews benchmark dataset.

CATT Benchmark Dataset				
Model	CE (%)		No CE (%)	
	DER	WER	DER	WER
EO – From Scratch	11.199	43.294	8.725	34.751
EO – MLM	10.037	39.904	7.919	32.598
ED – From Scratch	13.208	48.171	9.913	36.985
ED – MLM	11.345	42.841	8.805	34.066

Table 8: The impact of MLM pretraining vs. training from scratch, after training for *fewer* iterations when evaluated on CATT benchmark dataset.

WikiNews Benchmark Dataset				
Model	CE (%)		No CE (%)	
	DER	WER	DER	WER
EO – From Scratch	8.226	29.958	5.232	18.716
EO – MLM	6.915	26.221	4.263	16.120
ED – From Scratch	10.518	34.805	6.999	22.478
ED – MLM	8.681	29.224	5.741	18.617

Table 9: The impact of MLM pretraining vs. training from scratch, after training for *fewer* iterations when evaluated on WikiNews benchmark dataset.

CATT Benchmark Dataset				
Model	CE (%)		No CE (%)	
	DER	WER	DER	WER
EO + Long Training	9.613	38.685	7.631	31.610
+ MLM	9.260	37.492	7.411	30.969
+ NS	8.805	35.597	7.117	29.759
ED + Long Training	10.359	39.753	8.003	31.654
+ MLM	9.087	35.757	7.272	29.483
+ NS	8.624	34.191	6.989	28.477

Table 10: Performance comparison of our training techniques on the CATT benchmark dataset.

WikiNews Benchmark Dataset				
Model	CE (%)		No CE (%)	
	DER	WER	DER	WER
EO + Long Training	7.009	25.857	4.527	16.761
+ MLM	5.892	23.785	3.469	14.116
+ NS	5.425	22.132	3.105	12.679
ED + Long Training	7.271	24.870	4.464	14.202
+ MLM	6.376	21.954	4.001	12.926
+ NS	6.070	20.745	3.744	11.982

Table 11: Performance comparison of our training techniques on the WikiNews benchmark dataset.

5.3 RQ3: Is the NS approach effective in ATD models?

We tested the impact of the NS approach on both the EO and ED models as shown in Tables 10 and 11. Our results show that NS approach further improved both ED and EO models, showing a considerable reduction in both DER and WER. After evaluating on the CATT dataset, the ED model achieved the best overall performance. However, the EO model outperformed the ED model specifically in DER under all CE conditions when evaluated on the WikiNews dataset.

6 Conclusion

This paper proposed a new approach to training ATD models. The proposed approach was to initialize ATD models’ parameters from a pretrained character-based BERT model, then training the models for longer iterations. After that, we used the NS approach to further improve the performance of our models. We evaluated our approach by comparing it to 11 commercial and open-source models using two benchmark datasets: WikiNews and CATT. Our results show that our models outperformed all other models in both DER and WER. We open-source our CATT models and dataset for the research community to advance research in this area.

Limitations

Although this research advances the progress in the ATD task, it has some limitations. These limitations include:

- **Specific input assumption:** Our model is designed to work only with Arabic text. It does not handle numbers or special characters often found in real-world data. Filtering out these unwanted characters and numbers may alter the sentence structure, potentially resulting in incorrect ground truth diacritics. For example, consider the sentence اشتریت ٣ كتب (translation: I bought 3 books). If the number ٣ is removed, the sentence becomes اشتریت كتب, which results in an incorrect grammatical structure (correct structure: اشتریت كتباً) that may also lead to incorrect diacritization. A correct normalization should replace the numeral with an equivalent Arabic word. Meaning, the sentence should be transformed to اشتریت ثلاثة كتب, where ثلاثة represents the number 3 in Arabic. Therefore, it is suggested to have a normalization layer before the diacritizing text in a sequential pipeline.
- **No handling for partially diacritized input:** The models are not conditioned to process partially diacritized text, as we filter those diacritics out before the text is fed into the model.

References

- Hamza Abbad and Shengwu Xiong. 2020. [Multi-components system for automatic arabic diacritization](#). In *Advances in Information Retrieval*, pages 341–355, Cham. Springer International Publishing.
- Hamza Abbad and Shengwu Xiong. 2021. [Simple extensible deep learning model for automatic arabic diacritization](#). *Transactions on Asian and Low-Resource Language Information Processing*, 21(2):1–16.
- Bashar Al-Rfooh, Gheith Abandah, and Rami Al-Rfou. 2023. [Fine-tashkeel: Finetuning byte-level models for accurate arabic text diacritization](#).
- Badr AlKhamissi, Muhammad ElNokrashy, and Mohamed Gabr. 2020. [Deep diacritization: Efficient hierarchical recurrence for improved Arabic diacritization](#). In *Proceedings of the Fifth Arabic Natural Language Processing Workshop*, pages 38–48, Barcelona, Spain (Online). Association for Computational Linguistics.

- Manar M Almanea. 2021. [Automatic methods and neural networks in arabic texts diacritization: a comprehensive survey](#). *IEEE Access*, 9:145012–145032.
- Kareem Darwish, Ahmed Abdelali, Hamdy Mubarak, and Mohamed Eldesouki. 2021. [Arabic diacritic recovery using a feature-rich bilstm model](#). *Transactions on Asian and Low-Resource Language Information Processing*, 20(2):1–18.
- Kareem Darwish and Hamdy Mubarak. 2016. [Farasa: A new fast and accurate Arabic word segmenter](#). In *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC’16)*, pages 1070–1074, Portorož, Slovenia. European Language Resources Association (ELRA).
- Kareem Darwish, Hamdy Mubarak, and Ahmed Abdelali. 2017. [Arabic diacritization: Stats, rules, and hacks](#). In *Proceedings of the Third Arabic Natural Language Processing Workshop*, pages 9–17, Valencia, Spain. Association for Computational Linguistics.
- Ali Fadel, Ibraheem Tuffaha, Mahmoud Al-Ayyoub, et al. 2019a. [Arabic text diacritization using deep neural networks](#). In *2019 2nd international conference on computer applications & information security (ICCAIS)*, pages 1–7. IEEE.
- Ali Fadel, Ibraheem Tuffaha, Bara’ Al-Jawarneh, and Mahmoud Al-Ayyoub. 2019b. [Neural arabic text diacritization: State of the art results and a novel approach for machine translation](#). *CoRR*, abs/1911.03531.
- Asma Abdel Karim and Gheith Abandah. 2021. [On the training of deep neural networks for automatic arabic-text diacritization](#). *International Journal of Advanced Computer Science and Applications*, 12(8).
- Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. [Bert: Pre-training of deep bidirectional transformers for language understanding](#). In *Proceedings of NAACL-HLT*, pages 4171–4186.
- Jason Lee, Kyunghyun Cho, and Thomas Hofmann. 2017. [Fully character-level neural machine translation without explicit segmentation](#). *Transactions of the Association for Computational Linguistics*, 5:365–378.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. [Roberta: A robustly optimized bert pretraining approach](#). *arXiv preprint arXiv:1907.11692*.
- Ilya Loshchilov and Frank Hutter. 2018. [Decoupled weight decay regularization](#). In *International Conference on Learning Representations*.
- Mokthar Ali Hasan Madhfar and Ali Mustafa Qamar. 2020. [Effective deep learning models for automatic diacritization of arabic text](#). *IEEE Access*, 9:273–288.
- Abderrahman Skiredj and Ismail Berrada. 2024. [Arabic text diacritization in the age of transfer learning: Token classification is all you need](#).
- Yuxuan Wang, R.J. Skerry-Ryan, Daisy Stanton, Yonghui Wu, Ron J. Weiss, Navdeep Jaitly, Zongheng Yang, Ying Xiao, Zhifeng Chen, Samy Bengio, Quoc Le, Yannis Agiomyrgiannakis, Rob Clark, and Rif A. Saurous. 2017. [Tacotron: Towards End-to-End Speech Synthesis](#). In *Proc. Interspeech 2017*, pages 4006–4010.
- Qizhe Xie, Minh-Thang Luong, Eduard Hovy, and Quoc V Le. 2020. [Self-training with noisy student improves imagenet classification](#). In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10687–10698.
- Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. [Byt5: Towards a token-free future with pre-trained byte-to-byte models](#).
- Adams Wei Yu, Qihang Lin, Ruslan Salakhutdinov, and Jaime G. Carbonell. 2017. [Normalized gradient with adaptive stepsize method for deep neural network training](#). *CoRR*, abs/1707.04822.
- Taha Zerrouki and Amar Balla. 2017. [Tashkeela: Novel corpus of arabic vocalized texts, data for auto-diacritization systems](#). *Data in brief*, 11:147–151.